

AI_judge システム設計書

概要 / 技術構成 / システム構成図 / 画面遷移 / 画面項目 / API / DB / 実装順

1. 概要

項目	内容
アプリ名	AI_judge
概要	画像をアップロードするとAIが分類し、AIが注目した箇所をヒートマップで表示するWebアプリ。
目的	機械学習初心者が、画像分類AIの判定結果と注目箇所を視覚的に理解できるようにする。
対象ユーザー	機械学習初心者、友人・先生・就活先、制作者本人。
最初の完成目標	AI判定システムを作成し、モデルがどこに注目しているかをヒートマップ形式で表示できる状態にする。
後回しにするもの	ユーザーによるモデル作成、複数モデル比較、ランキング、ゲーム形式、ログイン機能、管理者画面。

2. baseline機能

機能	概要	扱い
画像アップロード	ユーザーが判定したい画像を送信する。	必須
AI画像判定	PyTorchモデルで画像分類を行う。	必須
判定結果表示	分類結果と信頼度を表示する。	必須
ヒートマップ表示	Grad-CAMでAIの注目箇所を表示する。	必須
判定履歴保存	画像情報、結果、日時をSQLiteに保存する。	推奨
初心者向け説明	AIが何をしているかを短く説明する。	推奨

3. 技術構成

分類	採用技術	理由
フロントエンド	HTML / CSS / JavaScript	Web画面、API通信、DOM操作の基本を学びやすい。
バックエンド	Python / FastAPI	PyTorchとの相性がよく、画像判定APIを作りやすい。
AIモデル	PyTorch	自作モデルや既存モデルを扱いやすい。
可視化	Grad-CAM	AIが注目した箇所をヒートマップとして表示できる。
DB	SQLite	小規模な履歴保存に向いており、導入が簡単。
画像保存	ローカル保存	初期開発では構成が単純で扱いやすい。

4. システム構成

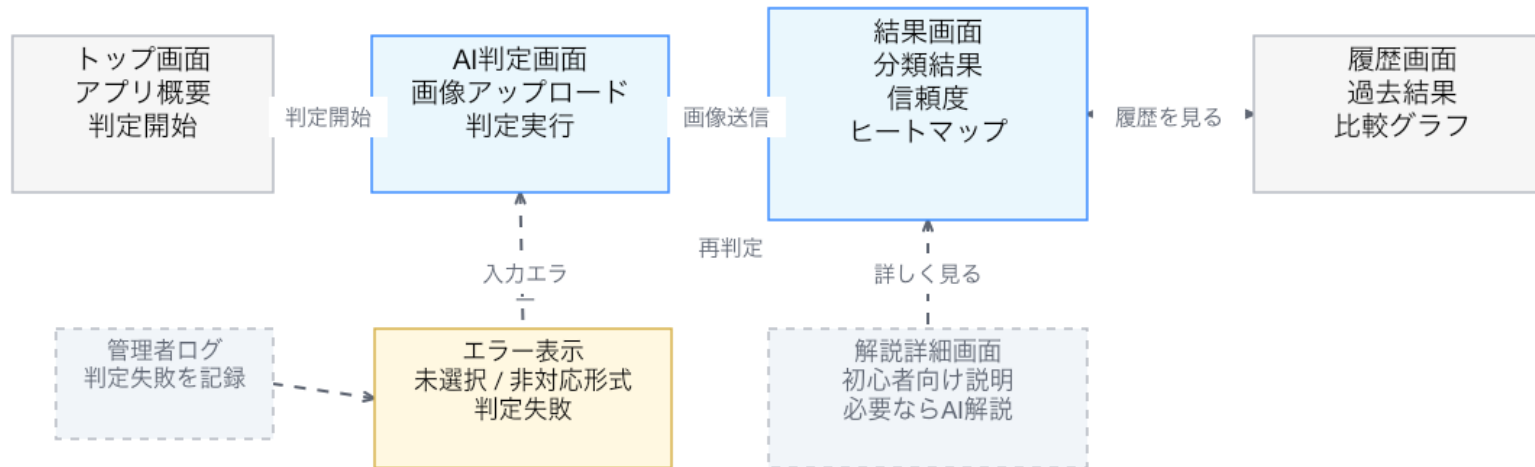
処理	流れ
画面操作	ユーザー -> HTML/CSS/JavaScript画面
画像判定	HTML/CSS/JavaScript画面 -> FastAPI -> PyTorchモデル
可視化	PyTorchモデル -> Grad-CAM -> ヒートマップ生成
保存	FastAPI -> SQLite / ローカル保存
表示	FastAPI -> HTML/CSS/JavaScript画面 -> 判定結果・ヒートマップ表示

5. 画面遷移図

主導線は「トップ画面 -> AI判定画面 -> 結果画面 -> 履歴画面」とし、解説詳細画面・管理者ログは追加候補として扱う。

baseline v1は判定体験を一直線に通す

主導線は短く保ち、詳しい解説・履歴・エラー対応は必要なときだけ開けるようにする。



実線: baselineの主導線 / 点線: 戻る・例外・追加候補

6. 画面別表示項目

画面	目的	表示項目	操作	遷移先	API	優先度
トップ画面	入口	アプリ名、説明、履歴リンク	判定開始、履歴を見る	AI判定、履歴	なし	必須
AI判定画面	画像選択と判定	画像選択、プレビュー、注意文、エラー領域	画像選択、判定	結果、エラー	POST /api/predictions	必須
結果画面	結果と注目箇所の確認	画像、分類結果、信頼度、ヒートマップ、説明	再判定、詳しく見る、履歴	AI判定、解説、履歴	GET /api/predictions/{id}	必須
解説詳細画面	初心者向け説明	分類説明、ヒートマップの読み方、注意点	結果へ戻る	結果	GET /api/predictions/{id}	推奨
履歴画面	過去結果の確認	日時、サムネイル、分類結果、信頼度	詳細、削除、戻る	結果、AI判定	GET /api/predictions	推奨
エラー表示	直せるエラーの案内	未選択、非対応形式、容量超過、判定失敗	選び直す、再判定	AI判定	POST /api/predictions	必須

7. API設計

分類	メソッド	エンドポイント	目的	リクエスト	レスポンス	優先度
判定	POST	/api/predictions	画像を受け取り、AI判定とGrad-CAM生成を行う。	image: file	prediction_id, class_name, confidence, image_url, heatmap_url, explanation	必須
判定	GET	/api/predictions/{id}	1件の判定結果を取得する。	path: prediction_id	prediction detail	必須
履歴	GET	/api/predictions	判定履歴の一覧を取得する。	limit, offset	predictions[]	推奨
履歴	DELETE	/api/predictions/{id}	不要な判定履歴を削除する。	path: prediction_id	deleted: true	推奨
モデル	GET	/api/models/current	現在使っているモデル情報を取得する。	なし	model_name, version, classes, description	推奨
説明	GET	/api/predictions/{id}/explanation	結果の詳しい説明を取得する。	path: prediction_id	beginner_explanation, heatmap_note	推奨
ログ	GET	/api/error-logs	判定失敗などのログを確認する。	limit, offset	error_logs[]	将来

8. DB設計

テーブル	カラム	型	内容	例	優先度
predictions	id	INTEGER / PK	判定結果ID	1	必須
predictions	original_image_path	TEXT	アップロード画像の保存先	uploads/20260716_xxx.png	必須
predictions	heatmap_image_path	TEXT	Grad-CAM画像の保存先	heatmaps/20260716_xxx.png	必須
predictions	class_name	TEXT	AIの分類結果	cat	必須
predictions	confidence	REAL	信頼度	0.92	必須
predictions	model_id	INTEGER	使用したモデルID	1	推奨
predictions	explanation	TEXT	初心者向けの簡単な説明	AIは顔付近に注目しています	推奨
predictions	created_at	TEXT	判定日時	2026-07-16 11:20:00	必須
models	id	INTEGER / PK	モデルID	1	推奨
models	name	TEXT	モデル名	ResNet50 baseline	推奨
models	version	TEXT	モデルのバージョン	v1	推奨
models	path	TEXT	モデルファイルの保存先	model/baseline/model.pth	推奨
models	description	TEXT	モデル説明	画像分類用のbaselineモデル	推奨
error_logs	id	INTEGER / PK	エラーログID	1	将来
error_logs	prediction_id	INTEGER	関連する判定ID	1	将来
error_logs	message	TEXT	エラー内容	unsupported file type	将来
error_logs	created_at	TEXT	エラー日時	2026-07-16 11:20:00	将来

9. 実装順メモ

順番	作業	完了条件	理由
1	画像アップロード画面を作る	HTMLから画像を選び、プレビューできる	最初にユーザー操作の入口を作る
2	FastAPIで画像を受け取る	POST /api/predictions が画像を保存できる	フロントとバックエンドをつなぐ
3	PyTorchモデルで判定する	class_name と confidence を返す	アプリの中心機能を作る
4	Grad-CAM画像を生成する	heatmap_image_path を返す	AIがどこを見たかを可視化する
5	結果画面を作る	画像、結果、ヒートマップを表示できる	baselineの価値が見える
6	SQLiteに保存する	履歴として判定結果を残せる	後で履歴画面につなげる
7	履歴画面を作る	過去の判定結果を一覧表示できる	学習・ポートフォリオとして見返せる